Final_Analysis

December 9, 2024

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
plt.style.use('ggplot')
```

[2]:
```
import ast

def transform_to_dict(unique_array):

    result_dict = {'0': 'Null'}

    for item in unique_array:
        if item != '0':
            variable_list = ast.literal_eval(item)

            for var in variable_list:
                # Diviser en numéro et nom, puis ajouter au dictionnaire
                num, name = map(str.strip, var.split(':'))
                result_dict[num] = name

    return result_dict
```

[4]:
```
def count_common_Motifs(df1, df2):

    common_counts = []

    unique_Supports = set(df1['Support']).intersection(set(df2['Support']))

    for Support in unique_Supports:
        compteur = 0
        Motifs1 = df1[df1['Support'] == Support]['Motifs'].iloc[0]
        Motifs2 = df2[df2['Support'] == Support]['Motifs'].iloc[0]  # Liste␣
 ↪d'ensembles

        for motif_2 in Motifs2:
            for motif_1 in Motifs1:
```

```
                if motif_2 == motif_1:
                    compteur  = compteur + 1

        common_counts.append({'Support': Support, 'Pattern': compteur})
    return common_counts
```

[5]:
```
def transform_column(df_column, variable_mapping):

    col_transform = []

    for x in df_column:
        t = x.strip("[]")
        a = t.split("}, {")

        result = [
            set(item.strip("{}").replace("'", "").split(", ")) for item in a
        ]

        transformed_line = []
        for group in result:
            new_group = set()
            for element in group:
                numero = ''.join(filter(str.isdigit, element))
                mapped_element = variable_mapping.get(numero, numero) +␣
↪(element[-1] if not numero == element else "")
                new_group.add(mapped_element)
            transformed_line.append(new_group)

        col_transform.append(transformed_line)

    return col_transform
```

[55]:
```
def plot_number_pattern(df, file_name ):

    plt.figure(figsize=(8, 4))
    sns.lineplot(
        data=df,
        x="Support",
        y="Size",
        hue="Dataset",
        markers=True,
        style="Dataset",
        linewidth=2,
        markersize=10,
    )

    plt.ylabel('Number of patterns')
```

```
    plt.xlabel('Minimum support threshold')

    plt.tight_layout()
    plt.savefig("../data/article/"+file_name+".eps")
    plt.savefig("../data/article/"+file_name+".jpeg")
    plt.show()
    plt.close()
```

[56]:
```
def plot_time(df, file_name ):

    plt.figure(figsize=(8, 5))
    sns.lineplot(
        data=df,
        x="Support",
        y="Temps d'exécution (s)",
        hue="Dataset",
        markers=True,
        style="Dataset",
        linewidth=2,
        markersize=10,
    )

    plt.ylabel("Time (s)")
    plt.xlabel('Minimum support threshold')

    plt.tight_layout()
    plt.savefig("../data/article/"+file_name+".eps")
    plt.savefig("../data/article/"+file_name+".jpeg")
    plt.show()
    plt.close()
```

[57]:
```
def plot_memory(df, file_name ):

    plt.figure(figsize=(8, 4))
    sns.lineplot(
        data=df,
        x="Support",
        y="Mémoire maximale (Ko)",
        hue="Dataset",
        markers=True,
        style="Dataset",
        linewidth=2,
        markersize=10,
    )

    plt.ylabel("Memory usage(Ko)")
    plt.xlabel('Minimum support threshold')
```

```
    plt.tight_layout()
    plt.savefig("../data/article/"+file_name+".eps")
    plt.savefig("../data/article/"+file_name+".jpeg")
    plt.show()
    plt.close()
```

[ ]:

```

```

0.1 Traitement du premier Dataset HCV

[7]:
```
hcv = pd.read_excel("../data/serveur/HCV.xlsx")
hcv_pca = pd.read_excel("../data/serveur/HCV_PCA.xlsx")
```

[8]:
```
df_hcv = pd.concat([hcv, hcv_pca], axis=0)
```

[38]:
```
df_hcv.shape
```

[38]: (39, 8)

[10]:
```
df_hcv["Dataset"] = np.where(df_hcv["Dataset"] == './data/AfterPCA/HCV.
↪csv','PCA HCV', 'HCV')
```

[13]:
```
hcv["Motifs"] = transform_column(hcv["Motifs detectés"],␣
↪transform_to_dict(hcv['Titre variables'].unique()))
hcv_pca["Motifs"] = transform_column(hcv_pca["Motifs detectés"],␣
↪transform_to_dict(hcv_pca['Titre variables'].unique()))
```

[14]:
```
hcv_valid = pd.DataFrame(count_common_Motifs(hcv, hcv_pca)).
↪sort_values('Support')
```

[15]:
```
hcv_valid
```

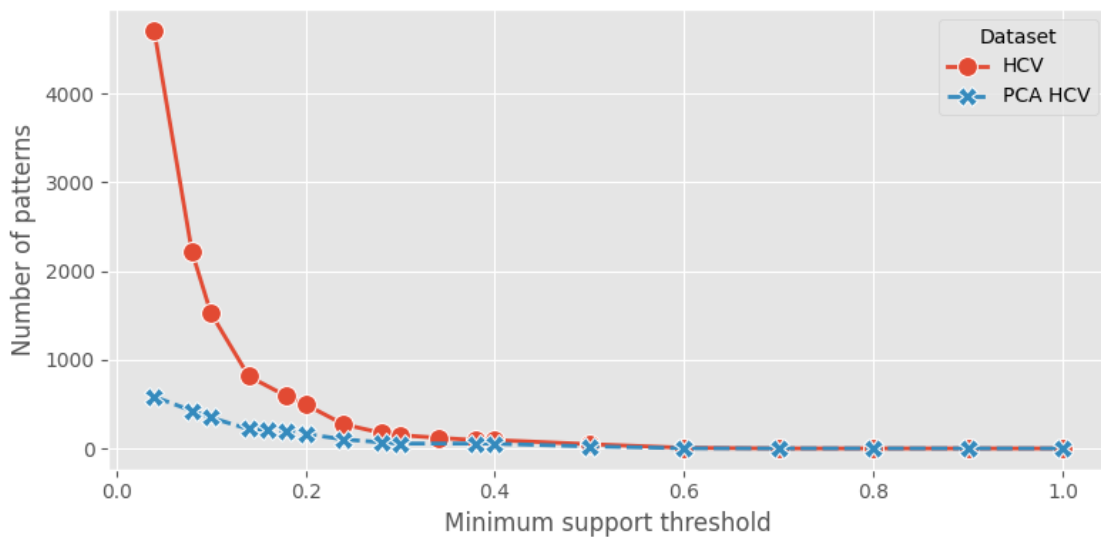
[15]:

	Support	Pattern
13	0.04	75
3	0.08	218
2	0.10	231
0	0.14	170
4	0.18	159
1	0.20	138
5	0.24	83
6	0.28	59
7	0.30	49
8	0.38	44
9	0.40	47
10	0.50	23

```
16      0.60        2
14      0.70        1
15      0.80        1
11      0.90        1
12      1.00        1
```

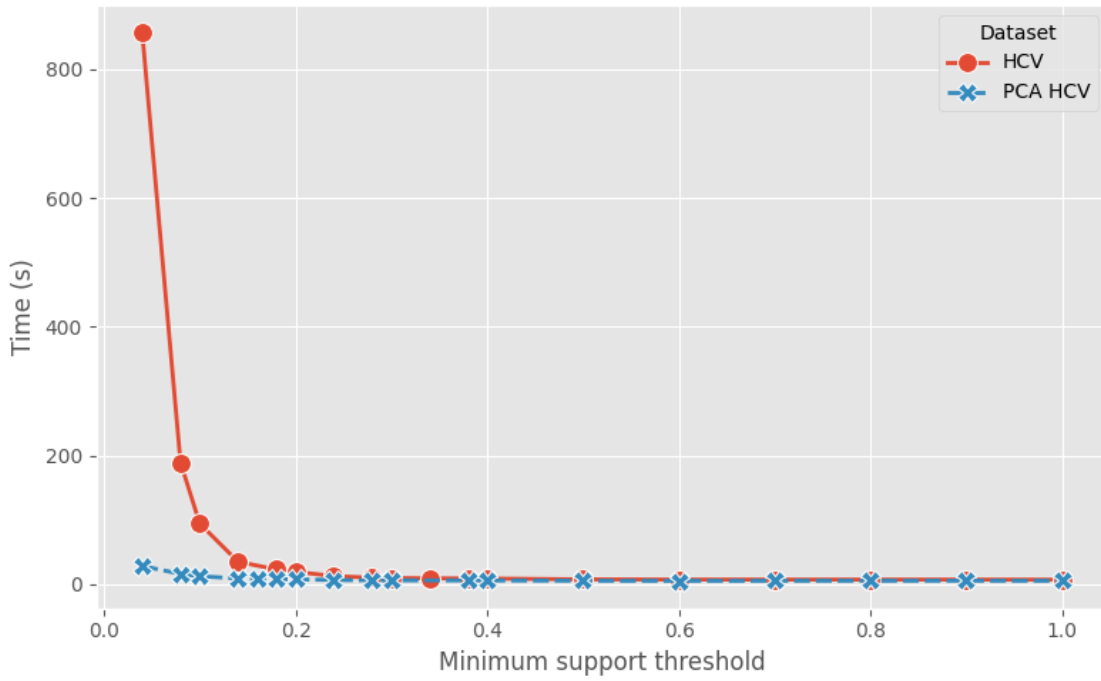
[26]:
```
plot_number_pattern(df_hcv, "HCV_support_vs_Nb_patters")
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



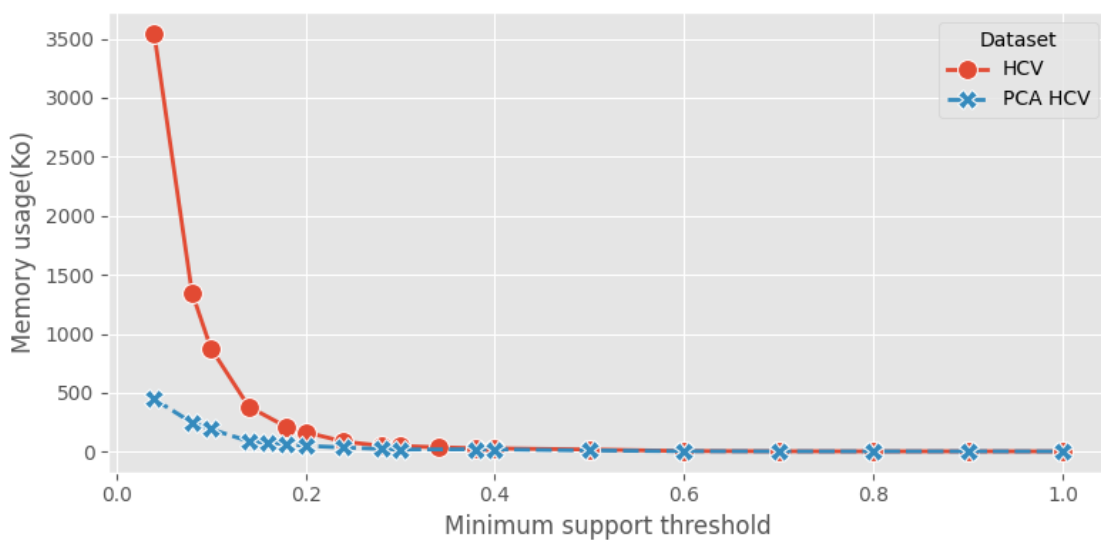
[33]:
```
plot_time(df_hcv,"HCV_Time_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



[35]:
```
plot_memory(df_hcv, "HCV_Memory")
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```

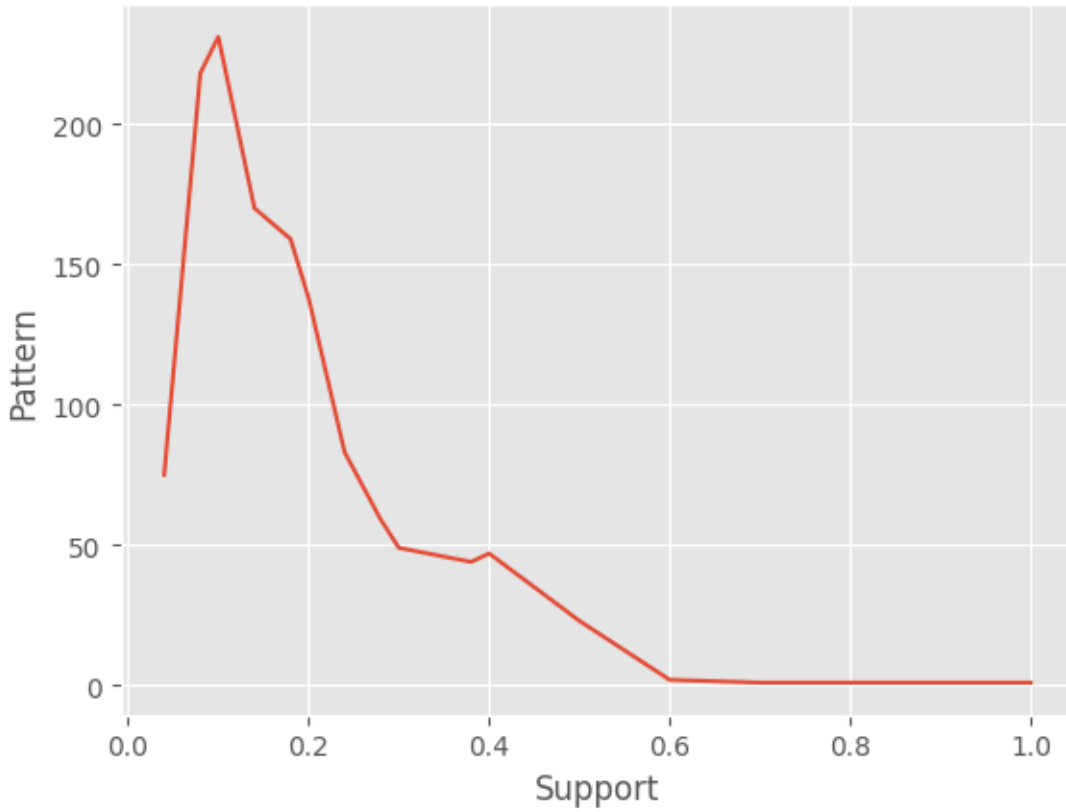


[ ]:
```

```

[36]:
```
sns.lineplot(data=hcv_valid, x="Support", y="Pattern")
```

[36]: <Axes: xlabel='Support', ylabel='Pattern'>



[ ]:
```

```

[ ]:
```

```

1 Traitement de Air quality dataset

[ ]:
```

```

[43]:
```
air = pd.read_excel("../data/serveur/Air Quality.xlsx")
air_pca = pd.read_excel("../data/serveur/Air Quality_PCA.xlsx")
```

[45]:
```
df_air = pd.concat([air, air_pca], axis=0)
df_air["Dataset"] = np.where(df_air["Dataset"] == './data/Clean/AirQualityUCI.
 ↪csv','Air Quality','PCA Air Quailty')
air["Motifs"] = transform_column(air["Motifs detectés"],␣
 ↪transform_to_dict(air['Titre variables'].unique()))
```

```
air_pca["Motifs"] = transform_column(air_pca["Motifs detectés"],␣
↪transform_to_dict(air_pca['Titre variables'].unique()))
air_valid = pd.DataFrame(count_common_Motifs(air, air_pca)).
↪sort_values('Support')
```

[46]:
```
air_valid
```

[46]:

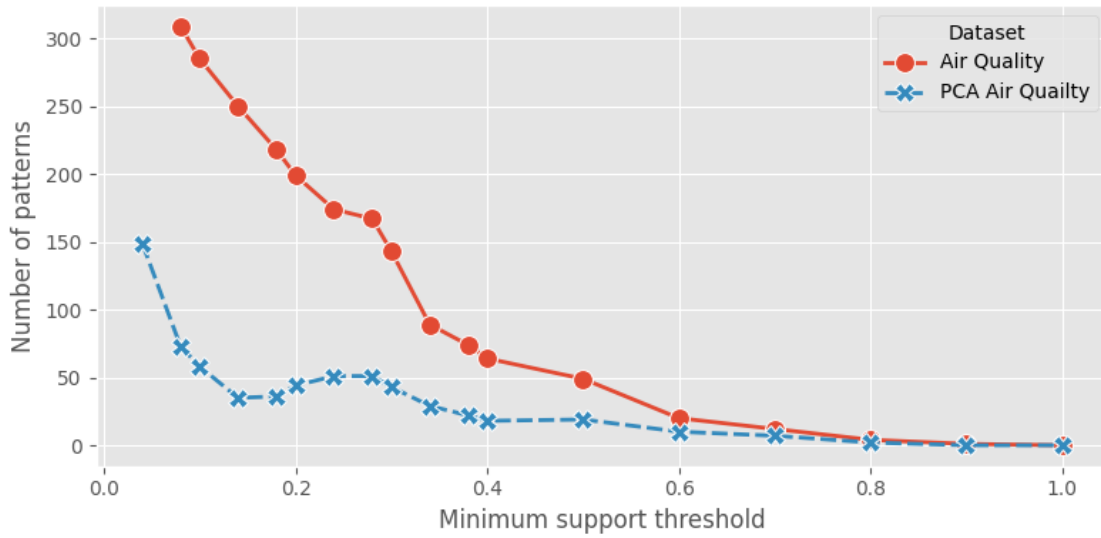
	Support	Pattern
0	0.08	14
3	0.10	13
1	0.14	6
2	0.18	4
4	0.20	7
5	0.24	11
6	0.28	11
7	0.30	10
8	0.34	9
9	0.38	7
10	0.40	7
11	0.50	4
16	0.60	4
14	0.70	3
15	0.80	1
12	0.90	0
13	1.00	1

[ ]:
```

```

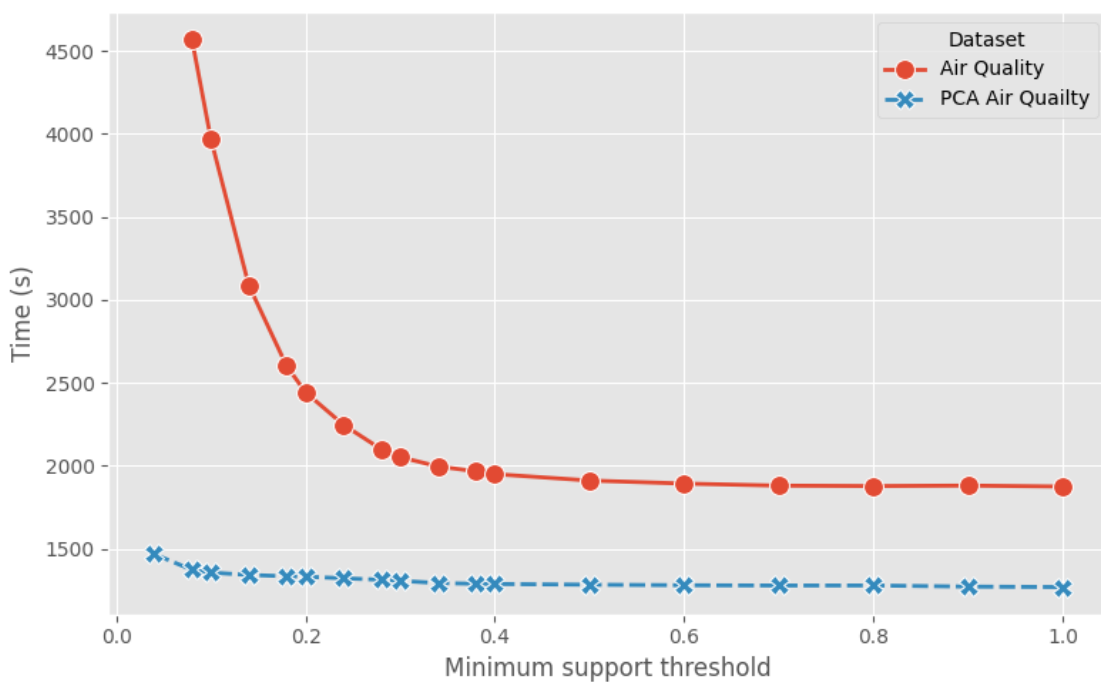
[70]:
```
plot_number_pattern(df_air, "AIR_support_vs_Nb_patters")
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



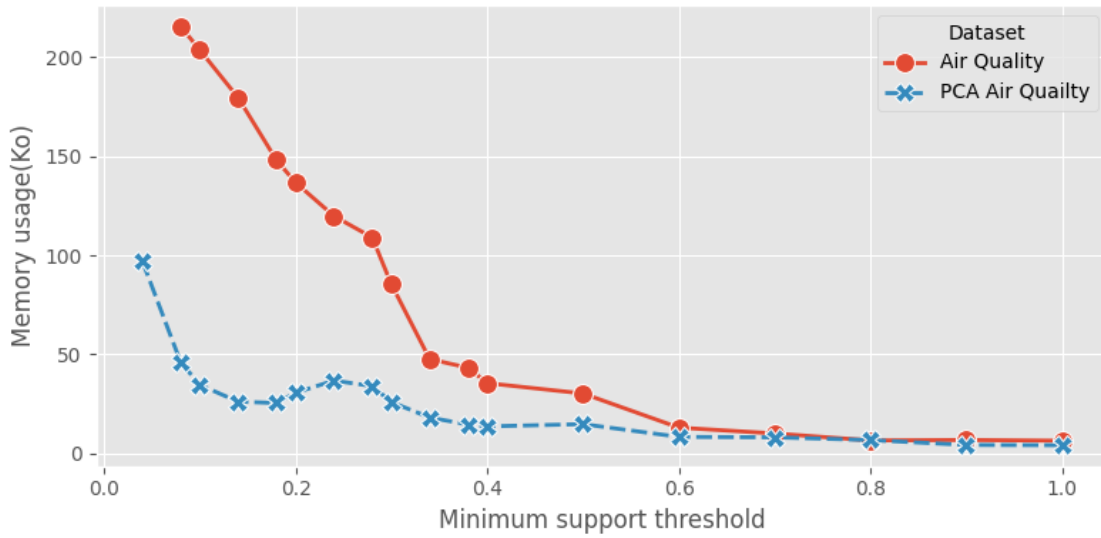
[59]:
```
plot_time(df_air,"AIR_Time_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



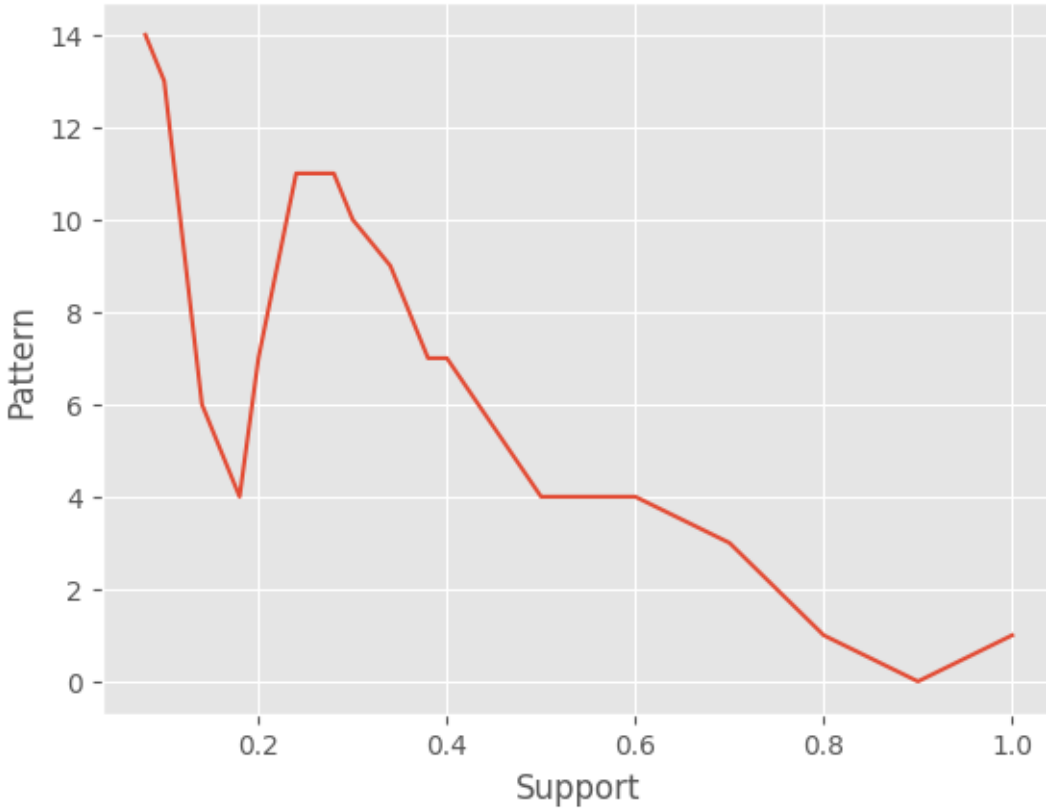
[60]:
```
plot_memory(df_air,"AIR_memory_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



[61]:
```
sns.lineplot(data=air_valid, x="Support", y="Pattern")
```

[61]:
```
<Axes: xlabel='Support', ylabel='Pattern'>
```



[ ]:

```
```

2 Traitement du dataset cargo

[ ]:

```
```

[63]:

```
cargo = pd.read_excel("../data/serveur/Cargo.xlsx")
cargo_pca = pd.read_excel("../data/serveur/Cargo_PCA.xlsx")
```

[64]:

```
cargo
```

[64]:

	Dataset	Support	Size	Temps d'exécution (s)
0	./data/Clean/Cargo_2000.csv	0.18	15112	53440.722686 \
1	./data/Clean/Cargo_2000.csv	0.20	11807	21149.035465
2	./data/Clean/Cargo_2000.csv	0.24	3624	2457.997721
3	./data/Clean/Cargo_2000.csv	0.28	1319	1501.517700
4	./data/Clean/Cargo_2000.csv	0.30	1039	1428.413661
5	./data/Clean/Cargo_2000.csv	0.34	803	1342.672671
6	./data/Clean/Cargo_2000.csv	0.38	815	1254.269561
7	./data/Clean/Cargo_2000.csv	0.40	775	1196.395916

```
8   ./data/Clean/Cargo_2000.csv     0.50    119             956.629695
9   ./data/Clean/Cargo_2000.csv     0.60     16             946.657938
10  ./data/Clean/Cargo_2000.csv     0.70      7            1021.741391
11  ./data/Clean/Cargo_2000.csv     0.80      4            1014.245546
12  ./data/Clean/Cargo_2000.csv     0.90      0             957.171214
13  ./data/Clean/Cargo_2000.csv     1.00      0             951.398436

    Mémoire maximale (Ko)                                   Titre variables
0             5448.187500  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…  \
1             3798.742188  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
2             1128.652344  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
3              381.625000  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
4              286.031250  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
5              224.070312  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
6              219.554688  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
7              209.210938  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
8               40.681641  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
9               13.483398  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
10              10.554688  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
11              10.786133  ['1 : i1_legid', '2 : i1_rcs_p', '3 : i1_rcs_e…
12               6.541992                                                 0
13               6.190430                                                 0

                                      Supports motifs
0   [0.21062073498872447, 0.21545343280665308, 0.2…  \
1   [0.21062073498872447, 0.21545343280665308, 0.2…
2   [0.42399144355396334, 0.4450517018462711, 0.44…
3   [0.4207548401324406, 0.42399144355396334, 0.44…
4   [0.4207548401324406, 0.42399144355396334, 0.44…
5   [0.4207548401324406, 0.42399144355396334, 0.44…
6   [0.4207548401324406, 0.42399144355396334, 0.50…
7   [0.4207548401324406, 0.42399144355396334, 0.50…
8   [0.5057330274002213, 0.5083164139345555, 0.504…
9   [0.8293391708316645, 0.7045439769836958, 0.643…
10  [0.8293391708316645, 0.7045439769836958, 0.837…
11  [0.8293391708316645, 0.837141211870524, 0.8620…
12                                                 0
13                                                 0

                                      Motifs detectés
0   [{'1+', '12+'}, {'1+', '12-'}, {'36+', '1+'}, …
1   [{'1+', '12+'}, {'1+', '12-'}, {'36+', '1+'}, …
2   [{'1+', '4-'}, {'1+', '13+'}, {'1+', '13-'}, {…
3   [{'1+', '4+'}, {'1+', '4-'}, {'1+', '6+'}, {'1…
4   [{'1+', '4+'}, {'1+', '4-'}, {'1+', '6+'}, {'1…
5   [{'1+', '4+'}, {'1+', '4-'}, {'1+', '6+'}, {'1…
6   [{'1+', '4+'}, {'1+', '4-'}, {'5+', '1+'}, {'1…
```

```
7   [{'1+', '4+'}, {'1+', '4-'}, {'5+', '1+'}, {'1…
8   [{'1+', '2+'}, {'3+', '1+'}, {'5+', '1+'}, {'1…
9   [{'3+', '2+'}, {'5+', '2+'}, {'7+', '2+'}, {'2…
10  [{'3+', '2+'}, {'5+', '2+'}, {'7+', '8+'}, {'1…
11  [{'3+', '2+'}, {'7+', '8+'}, {'27+', '26+'}, {…
12                                                 0
13                                                 0
```

[65]:
```
df_cargo = pd.concat([cargo, cargo_pca], axis=0)
df_cargo["Dataset"] = np.where(df_cargo["Dataset"] == './data/Clean/Cargo_2000.
 ↪csv','Cargo 2000','PCA Cargo 2000')
cargo["Motifs"] = transform_column(cargo["Motifs detectés"],␣
 ↪transform_to_dict(cargo['Titre variables'].unique()))
cargo_pca["Motifs"] = transform_column(cargo_pca["Motifs detectés"],␣
 ↪transform_to_dict(cargo_pca['Titre variables'].unique()))
cargo_valid = pd.DataFrame(count_common_Motifs(cargo, cargo_pca)).
 ↪sort_values('Support')
```

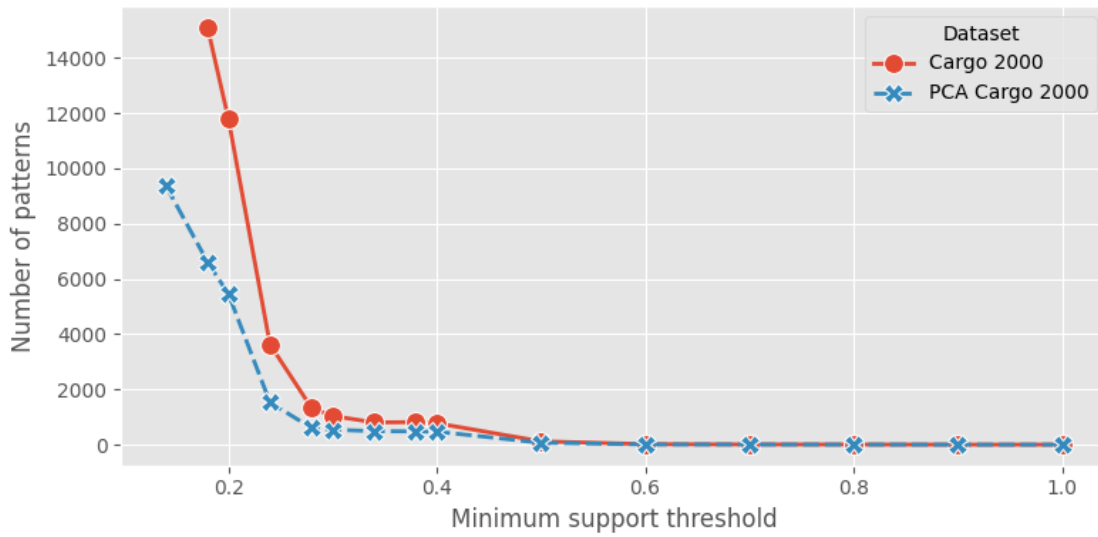
[67]:
```
cargo_valid
```

[67]:

	Support	Pattern
0	0.18	161
3	0.20	165
1	0.24	715
2	0.28	467
4	0.30	413
5	0.34	387
6	0.38	420
7	0.40	419
8	0.50	55
9	0.60	7
12	0.70	4
13	0.80	2
10	0.90	1
11	1.00	1

[71]:
```
plot_number_pattern(df_cargo, "CARGO_support_vs_Nb_patters")
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



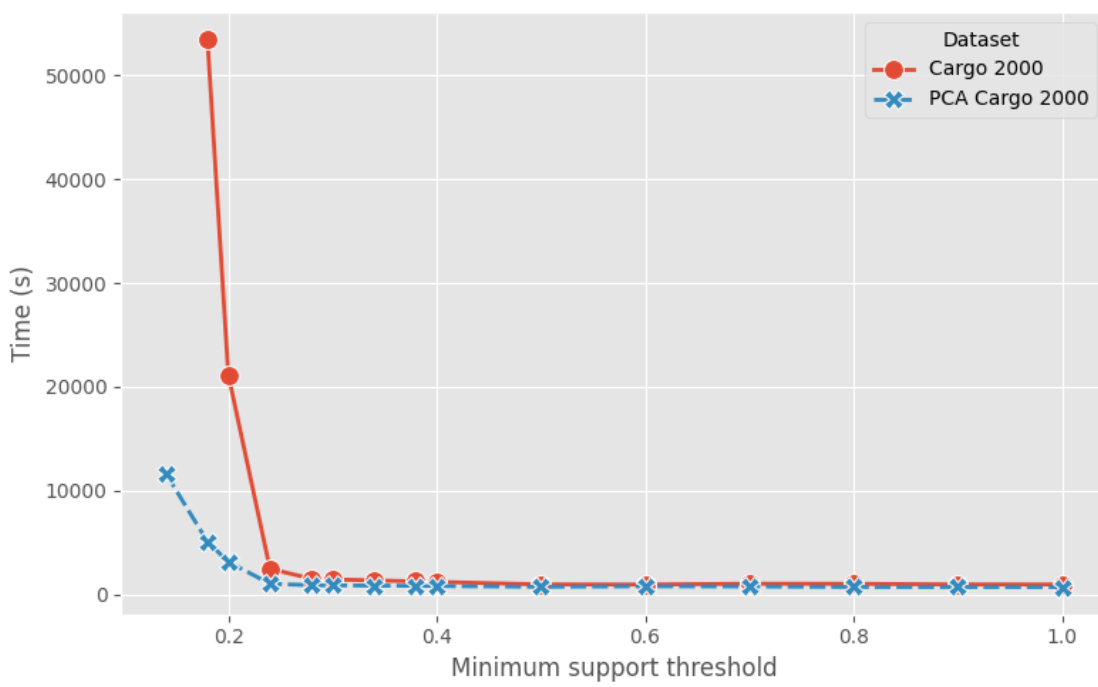
[ ]:

```
```

[72]:

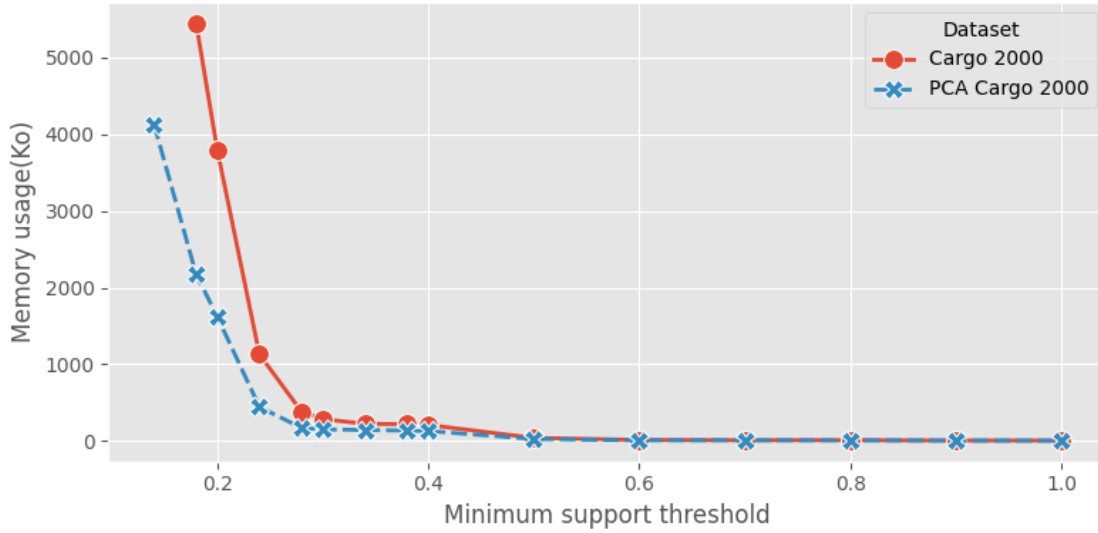
```
plot_time(df_cargo,"CARGO_Time_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



[74]:
```
plot_memory(df_cargo,"CARGO_memory_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



[ ]:
```

```

3 Traitement du dataset Chickenpox

[76]:
```
chicken = pd.read_excel("../data/serveur/Chickenpox.xlsx")
chicken_pca = pd.read_excel("../data/serveur/Chickenpox_PCA.xlsx")
```

[77]:
```
chicken.head()
```

[77]:
```
                        Dataset  Support   Size  Temps d'exécution (s)
0  ./data/Clean/Chickenpox.csv     0.34  41522          172638.235327  \
1  ./data/Clean/Chickenpox.csv     0.38  22932           30516.866726
2  ./data/Clean/Chickenpox.csv     0.40  16067           10529.315486
3  ./data/Clean/Chickenpox.csv     0.50   2280             144.305780
4  ./data/Clean/Chickenpox.csv     0.60    386              17.552623

   Mémoire maximale (Ko)                                   Titre variables
0           31800.433594  ['1 : BUDAPEST', '2 : BARANYA', '3 : BACS', '4…  \
1           17329.897461  ['1 : BUDAPEST', '2 : BARANYA', '3 : BACS', '4…
2           12136.811523  ['1 : BUDAPEST', '2 : BARANYA', '3 : BACS', '4…
3            1281.600586  ['1 : BUDAPEST', '2 : BARANYA', '3 : BACS', '4…
4             104.405273  ['1 : BUDAPEST', '2 : BARANYA', '3 : BACS', '4…
```

```
                                        Supports motifs
0  [0.34549679734668814, 0.35096079599355795, 0.3…  \
1  [0.39770262021900116, 0.39309168192615146, 0.3…
2  [0.42511086107617974, 0.42213250380567874, 0.4…
3  [0.5417815724255595, 0.5384134548208941, 0.525…
4  [0.6669681793780013, 0.6727925224847588, 0.681…

                                        Motifs detectés
0  [{'7+', '4+', '17+', '6+', '1+', '18+'}, {'4+'…
1  [{'13+', '1+', '18+', '4+', '6+'}, {'1+', '18+…
2  [{'13+', '4+', '18+', '6+'}, {'16+', '4+', '18…
3  [{'18+', '1+', '4+'}, {'18+', '1+', '16+'}, {'…
4  [{'2+', '18+'}, {'2+', '20+'}, {'3+', '17+'}, …
```

[ ]:

```
```

[ ]:

```
```

[78]:

```
df_chicken = pd.concat([chicken, chicken_pca], axis=0)
df_chicken["Dataset"] = np.where(df_chicken["Dataset"] == './data/Clean/
 ↪Chickenpox.csv','Chickenpox','PCA Chickenpox')
chicken["Motifs"] = transform_column(chicken["Motifs detectés"],␣
 ↪transform_to_dict(chicken['Titre variables'].unique()))
chicken_pca["Motifs"] = transform_column(chicken_pca["Motifs detectés"],␣
 ↪transform_to_dict(chicken_pca['Titre variables'].unique()))
chicken_valid = pd.DataFrame(count_common_Motifs(chicken, chicken_pca)).
 ↪sort_values('Support')
```

[79]:

```
chicken_valid
```

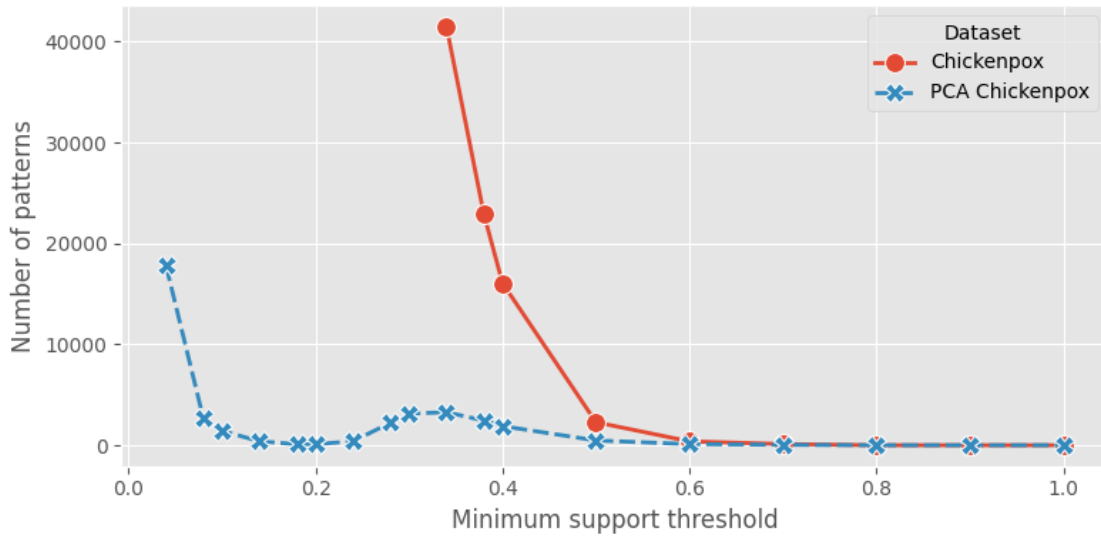
[79]:

	Support	Pattern
0	0.34	367
1	0.38	503
4	0.40	247
3	0.50	287
2	0.60	92
5	0.70	44
6	0.80	0
7	0.90	1
8	1.00	1

[81]:

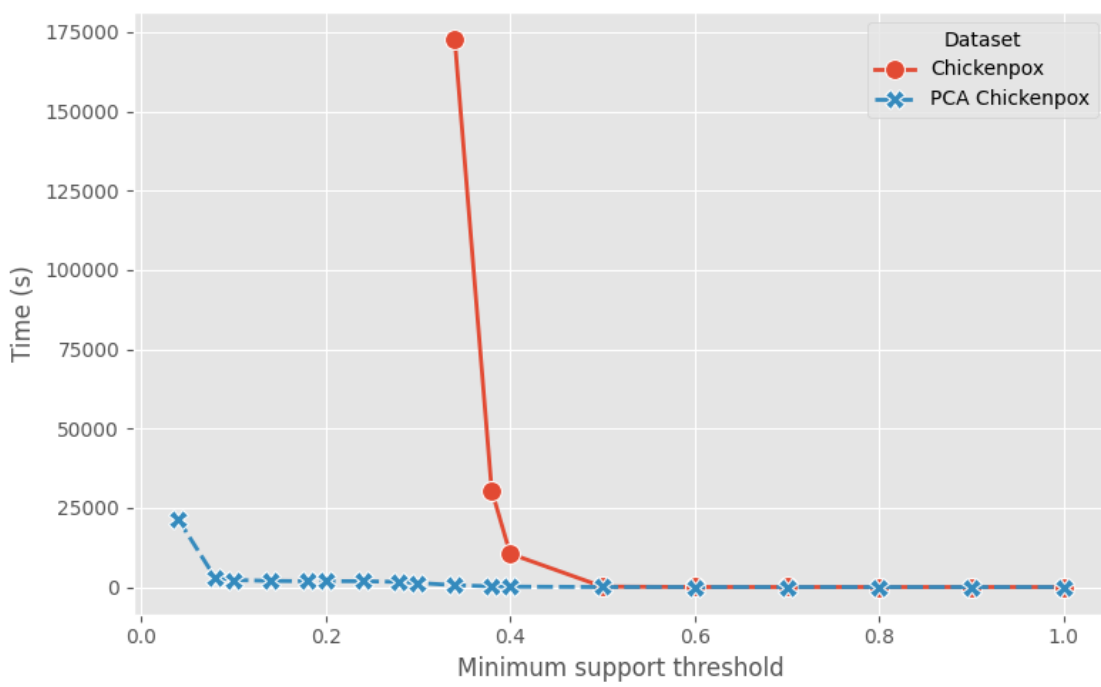
```
plot_number_pattern(df_chicken, "CHICKEN_support_vs_Nb_patters")
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



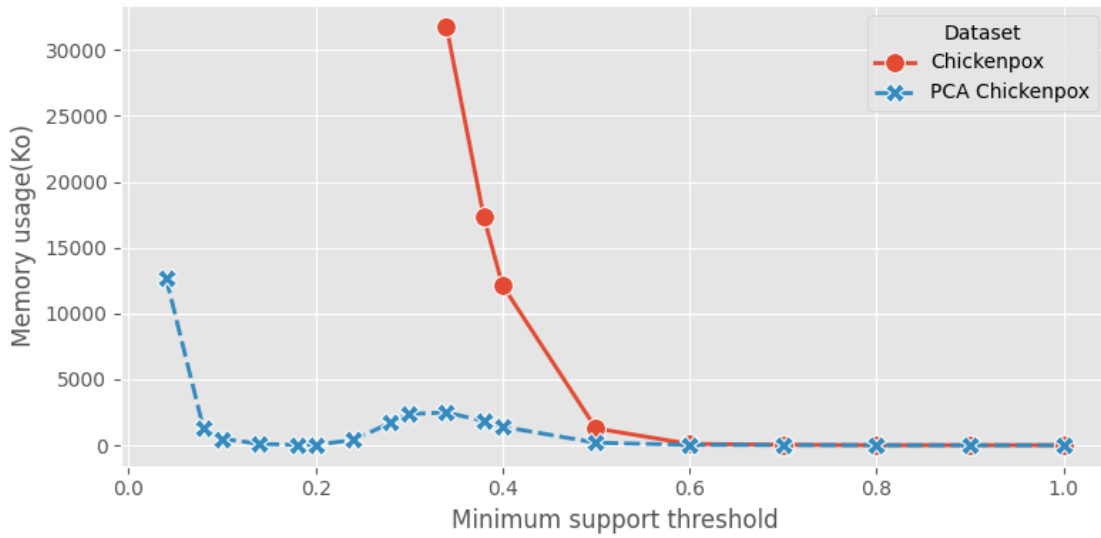
[82]:
```
plot_time(df_chicken,"CHICKEN_Time_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```



[83]:
```
plot_memory(df_chicken,"chicken_memory_vs_support" )
```

```
The PostScript backend does not support transparency; partially transparent
artists will be rendered opaque.
```




```
[ ]:
```